

AI AGENT 项目实战营 · 第 05 节

MCP Server

接现成 Server，跑通真实任务

本节少讲协议，多做一条用法：选 Server → 限权限 → 接 Agent → 看输出 → 再自建一把工具。

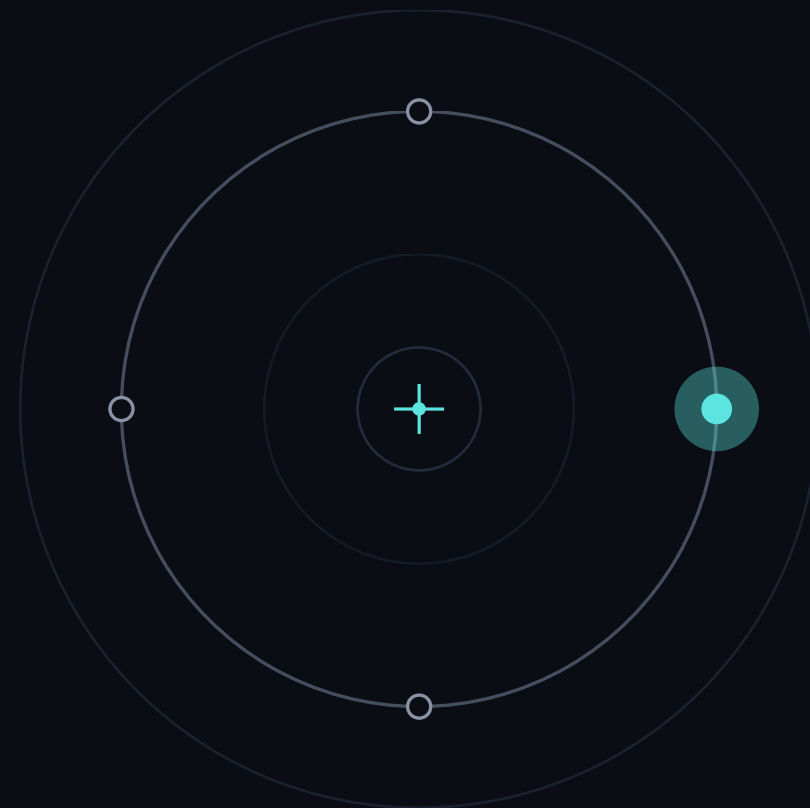
PROJECT · ResuMatch · LECTURE + LAB · 60 MIN



00

Part 0

今天怎么用



- ✓ 今天不把 MCP 当协议背，而是当一套接能力的方法用。
- ✓ 先接官方文件 Server，真读一份简历。
- ✓ 再把 ResuMatch 的打分逻辑，发布成自己的 Server。
- ✓ 最后用清单判断：该接现成、该自建、该不用 MCP。

今天的目标：能接、能跑、能限制权限、能截图交付。

80

第 4 节：为读一份简历
手写 80 行文件 I/O



1

第 5 节：接一个文件 Server
一行读文件，直连真实简历

使用价值先看这里：读文件这类通用能力，直接接现成。

第 4 节的做法

- 要读一份简历文件——
- 打开、判后缀、解析、抽文本……
- 80 行之后你只是「读了个文件」
- 下周要支持新格式，再重写一遍

今天的用法

- ✓ 接官方文件 Server
- ✓ 给它一个安全目录
- ✓ Agent 调 read_file 读简历
- ✓ 拿到真实输出截图

先把一条真实链路跑通，再讲为什么这样设计。

P1 先判断：这个项目哪里需要 MCP。

P2 够用概念：只学今天要用到的几个名词。

P3 接现成：官方文件 Server 读简历。

P4 自己做：把打分逻辑发布成 Server。

P5 用得安全：权限、投毒、失败降级。

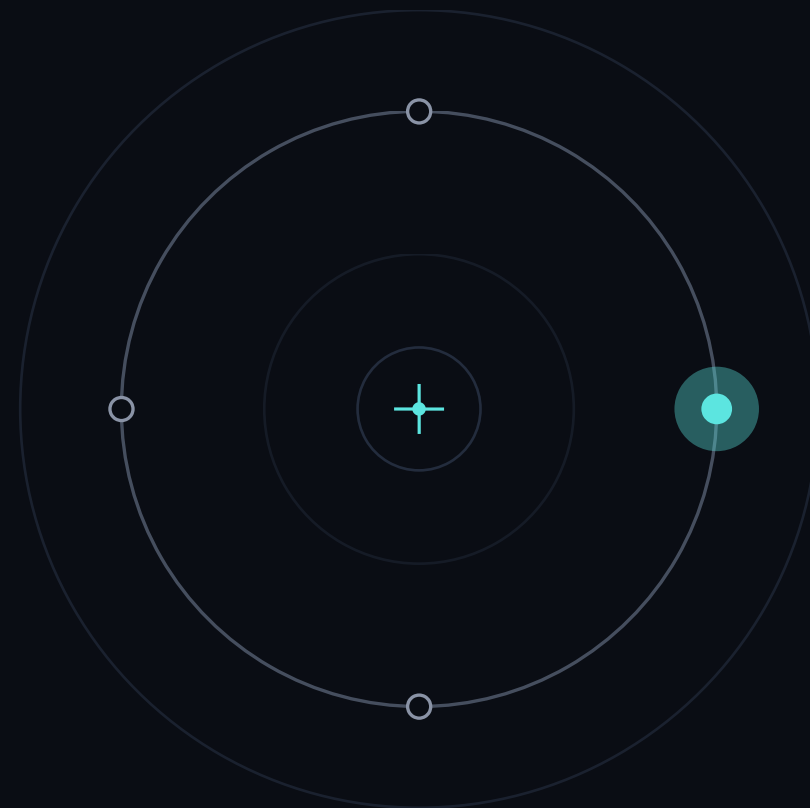
P6 交作业：按清单跑通并截图。

每一站都回答一个问题：我现在该怎么用。

01

Part 1

使用场景



- ✓ ① 做真事、能验证——真的接上 Server，真的调通。
- ✓ ② 处理真实数据——真读真文件、真调真服务。
- ✓ ③ 不编造——答案来自真实来源，不瞎编。
- ✓ ④ 会失败还不崩——Server 连不上，能降级不崩溃。

先按这四条验收，不按名词验收。

RESUMATCH

求职助手

接文件 Server 读真实简历，
对着 JD 打分、报缺口。

COURSE RAG

课程问答助手

接文档 / 检索 Server，
答案必须来自检索材料，
并标明出处。

SCHEDULE

个人日程助手

接日历 / 文件 Server，
真读日程、真建事件、
真提醒。

先问：我的项目要接哪种真实能力。

先 CONNECT

- ✓ 文件、日历、数据库这类通用能力
- ✓ 优先接官方或可信 Server
- ✓ 先跑出结果，不急着自建
- ✓ 今天先接文件 Server

再 BUILD

- ✓ 打分规则、业务流程、课程标准
- ✓ 属于你的专有逻辑
- ✓ 写成自己的 Server
- ✓ 后续多个客户端复用

使用顺序：通用能力先接现成，专有逻辑再自建。

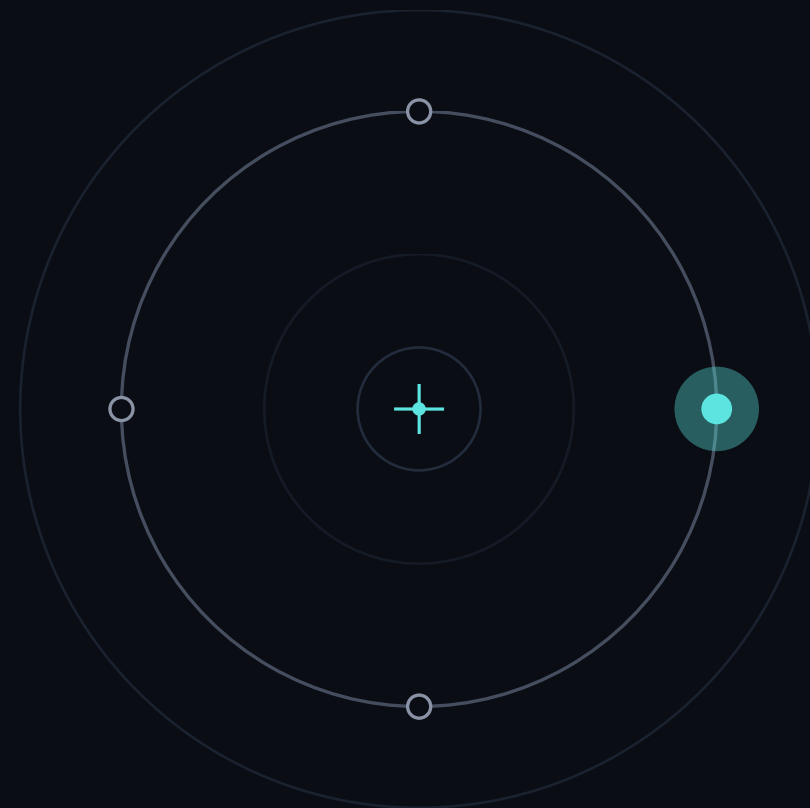
- ✓ 只是一个小函数、只给当前 Agent 用：先用本地 tool。
- ✓ 能力已经有人维护好：接现成 MCP Server。
- ✓ 你的能力要给多个地方复用：做成自己的 MCP Server。

这页是今天最重要的判断题：不要什么都 MCP 化。

02

Part 2

够用概念



$M \times N$

没有统一接法：
每个应用 $\times$ 每个工具都要单独接



$M + N$

有了 MCP：
应用按一种方式接，能力按一种方式暴露

够用理解：MCP 是 Agent 接外部能力的标准接法。

HOST

你在用的应用

AI 应用本体，
你的 ResuMatch 程序。
它负责发起任务。

CLIENT

SDK 帮你接线

Host 里的连接器。
通常不用手写，
但要知道它一对一连 Server。

SERVER

外部能力

真正提供工具 / 数据
的小程序，
比如官方文件 Server。

使用时只要分清：我写 Host，SDK 接线，Server 提供能力。

TOOLS

先看 Tools

它能做什么动作。
读文件、查库、调 API、打分。

RESOURCES

再看 Resources

它能提供什么只读数据。
文件内容、评分细则、配置。

PROMPTS

Prompts 可后看

预设 workflow 或模板。
入门先不用展开。

读一个 Server 文档时，先找 Tools，再找权限和启动方式。

今天用 stdio

- ✓ 本机启动 Server 子进程
- ✓ 命令和参数写在代码里
- ✓ 最快、最少配置
- ✓ 适合课堂和本地项目

以后再用 HTTP

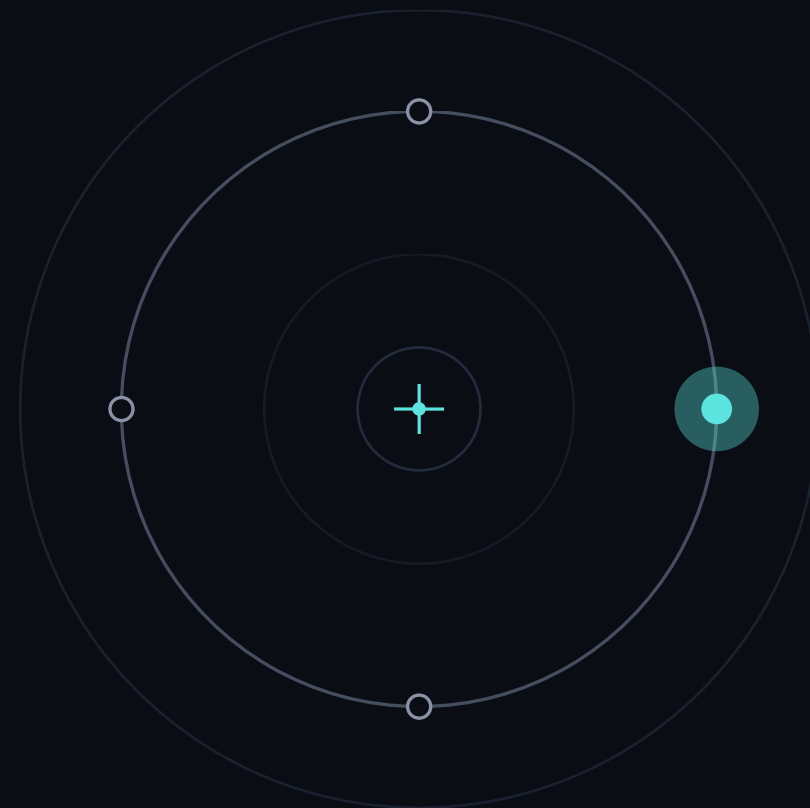
- ✓ Server 部署在远端
- ✓ 多人或多客户端共用
- ✓ 需要鉴权和运维
- ✓ 上线后再处理

本节只用 stdio：先跑通本机，再谈远程。

03

Part 3

接一个 Server



第 1 步：选官方文件 Server，把目录当沙箱。

```
# start the official filesystem server on your machine
npx -y @modelcontextprotocol/server-filesystem ./sample_docs

# it now listens; the Agent launches it as a child process
```

使用前先定边界

- ✓ -y = 跳过确认
- ✓ 目录只放本次要读的文件
- ✓ 越界文件不要放进去

第一原则：先给 Server 一个小沙箱，再让 Agent 用。

第 2 步：把启动命令写进 MCPServerStdio。

```
from pathlib import Path
from agents import Agent, Runner
from agents.mcp import MCPServerStdio

resume_dir = Path(__file__).parent / "sample_docs"

async with MCPServerStdio(
    name="Filesystem Server",
    params={"command": "npx", "args":
        ["-y", "@modelcontextprotocol/server-filesystem",
        str(resume_dir)]},
) as server:
    ...
```

照抄这三件事

- ✓ name = 你给它的名字
- ✓ command = 怎么启动
- ✓ args = 启动参数
- ✓ 退出时自动清理

第 3 步：只放行今天需要的工具。

```
from agents.mcp import (MCPServerStdio,
                        create_static_tool_filter)

resume_server = MCPServerStdio(
    params={...}, # same npx command as before
    tool_filter=create_static_tool_filter(
        allowed_tool_names=
            ["read_file", "list_directory"],
    ), # write/delete are now blocked
)
```

最小权限落地

- ✓ 只放 read_file
- ✓ 只放 list_directory
- ✓ 其它写入 / 删除先不开放

只开放 read_file 和 list_directory，先挡掉写入与删除。

第 4 步：把活着的 Server 放进 Agent。

```
agent = Agent(  
    name="ResuMatch",  
    instructions=(  
        "Read resume.md and jd.txt from the dir. "  
        "List matched skills and gaps. "  
        "Use only the files; never invent."),  
    mcp_servers=[server],    # the entire wiring  
)  
result = await Runner.run(agent,  
    "Read resume.md and jd.txt; matches and gaps?")
```

不用逐个包装工具

- ✓ mcp_servers=[server]
- ✓ SDK 自动列工具
- ✓ 红线写进 instructions
- ✓ 只依据文件，绝不编造

第 5 步：看 trace ，确认它真的读了文件。

```
# ---- output ----  
# [Agent] tool: list_directory("./sample_docs")  
# [Agent] tool: read_file("./sample_docs/resume.md")  
# [Agent] tool: read_file("./sample_docs/jd.txt")  
#  
# Matched: Python, SQL, Docker (JD wants, resume has)  
# Missing: AWS, Kubernetes (add a cloud project)
```

截图交付看三样

- ✓ 调用了 list_directory
- ✓ 调用了 read_file
- ✓ 输出里有匹配和缺口
- ✓ 没有编造外部信息

跑通不是看它会说，而是看它真的调了 Server 。

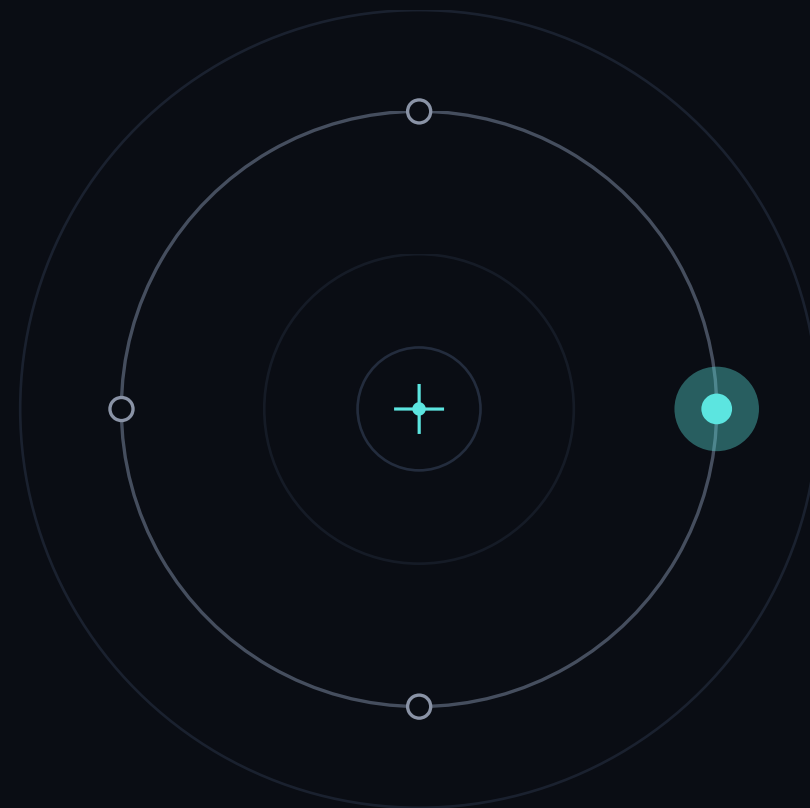
- ✓ ① 选可信 Server：官方优先。
- ✓ ② 定边界：只给本次需要的目录和权限。
- ✓ ③ 接进 Agent：mcp_servers=[server]。
- ✓ ④ 看 trace：确认真的调了工具。
- ✓ ⑤ 留截图：真实输出就是交付证据。

这张清单比概念定义更重要。

04

Part 4

自己做 Server



只有专有逻辑才自建；安装口径用 `mcp[cli]`。

```
# install: uv add "mcp[cli]"    (not the community fastmcp)

# resumatch_server.py
from mcp.server.fastmcp import FastMCP

mcp = FastMCP("resumatch")    # the display name hosts show
```

自建只做三件事

- ✓ 起 Server
- ✓ 挂工具
- ✓ 用 Inspector 试调

先起一个最小 Server，不要一开始就做大。

第 1 把工具：把 JD 文本变成结构化字段。

```
@mcp.tool()
def parse_jd(jd_text: str) → dict:
    """Parse a job description into fields."""
    bank = ["python", "sql", "aws", "docker"]
    text = jd_text.lower()
    return {"required_skills":
            [s for s in bank if s in text]}
# parse_jd("Senior Python, AWS, Docker")
# → {"required_skills": ["python", "aws", "docker"]}
```

写工具时盯三样

- ✓ 名字要像动作
- ✓ docstring 说明何时用
- ✓ 类型注解写清参数
- ✓ 返回结构化结果

第 2 把工具：用确定代码算分，不交给模型猜。

```
@mcp.tool()
def score_resume(resume_text: str,
                 required_skills: list[str]) → dict:
    """Score a resume vs required skills (0-100)."""
    text = resume_text.lower()
    hit = [s for s in required_skills if s in text]
    miss = [s for s in required_skills if s not in text]
    return {"match_score":
            round(100*len(hit)/len(required_skills)),
            "matched": hit, "missing": miss}
```

这类逻辑适合自建

- ✓ 评分规则你掌控
- ✓ 输入输出可测试
- ✓ 结果可复现
- ✓ 后面多个客户端复用

接进 Agent 之前，先用 Inspector 单独试工具。

```
# resumatch_server.py end -- never print() to stdout
if __name__ == "__main__":
    mcp.run(transport="stdio")

# terminal: uv run mcp dev resumatch_server.py
# ---- output ----
# MCP Inspector at http://localhost:6274
# Tools: parse_jd, score_resume
# > parse_jd {"jd_text":"Python, AWS"} → [python, aws]
```

先单测，再接入

- ✓ mcp dev 打开 Inspector
- ✓ 填参数，看返回
- ✓ 工具 OK 再接 Agent
- ✓ 调试日志写 stderr

把评分标准也做成只读数据，避免到处复制。

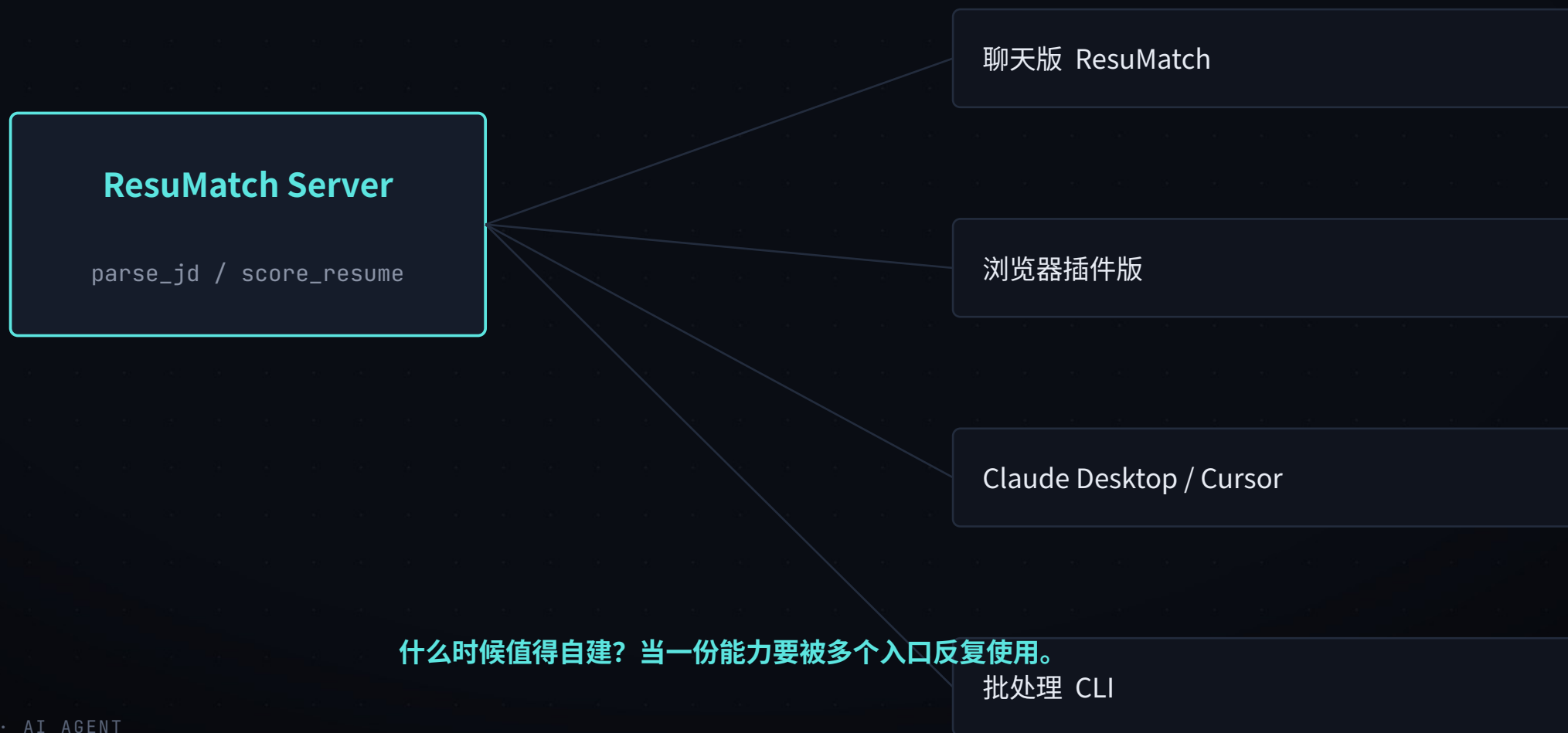
```
@mcp.resource("rubric://{role}")
def get_rubric(role: str) → str:
    """Return the scoring rubric for a role."""
    return (f"Rubric for {role}: skills 60%, "
           f"experience 30%, impact 10%.")

# any client can now read: rubric://data-analyst
```

Resource 的使用场景

- ✓ 只读标准
- ✓ 岗位不同，URI 不同
- ✓ 任何客户端都能读
- ✓ 标准只维护一份

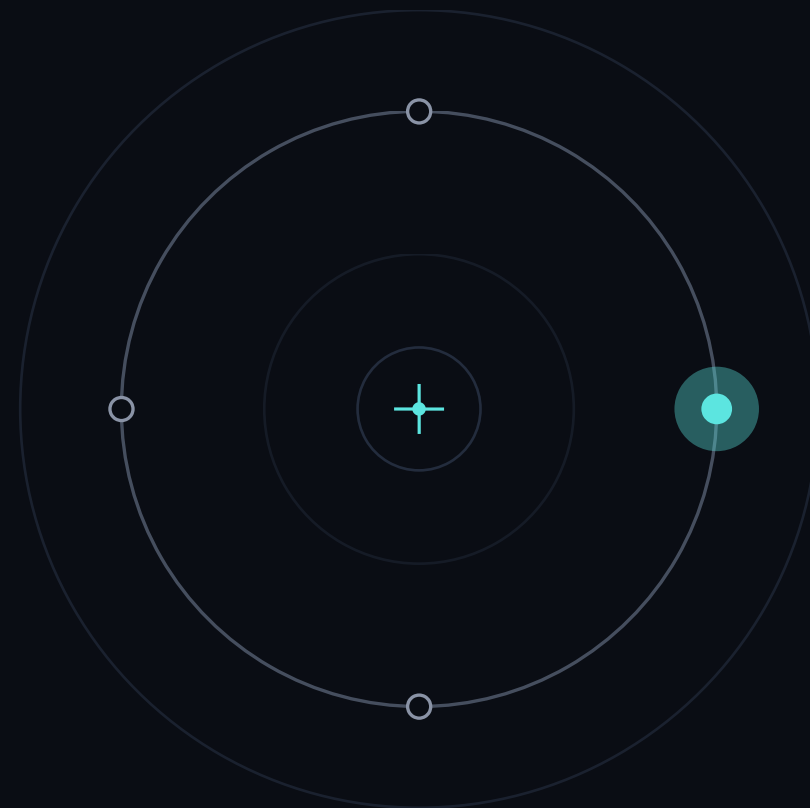
工具负责做事，Resource 负责给标准。



05

Part 5

用得安全



本地函数工具

- 一只给当前 Agent 用
- 一逻辑很小
- 一不用单独授权
- 一要快速调试

MCP Server

- ✓ 要接别人维护的能力
- ✓ 要给多个客户端复用
- ✓ 要独立权限边界
- ✓ 要以后能替换实现

使用判断：能简单就简单，需要边界和复用才上 MCP。

自建 Server

- ✓ parse_jd / score_resume
- ✓ 评分标准
- ✓ 改简历模板
- ✓ 这些是你的产品逻辑

接现成 Server

- ✓ 读文件
- ✓ 列目录
- ✓ 以后查日历 / 查库
- ✓ 这些是通用能力

一个项目可以混用：通用的接，专有的建。

- ✓ 使用别人 Server 前，不只看名字，要看完整工具描述。
- ✓ 模型会认真读工具描述，人常常只扫一眼工具名。
- ✓ Server 返回的内容也当不可信：网页、邮件、文件都可能藏指令。
- ✓ 所以：可信来源 + 最小权限 + 真实 trace ，三件事一起做。

安全不是额外章节，是每次接 Server 的使用动作。

接入前检查

- ✓ 来源可信
- ✓ 工具描述读完
- ✓ 权限最小
- ✓ 沙箱够小

运行时兜底

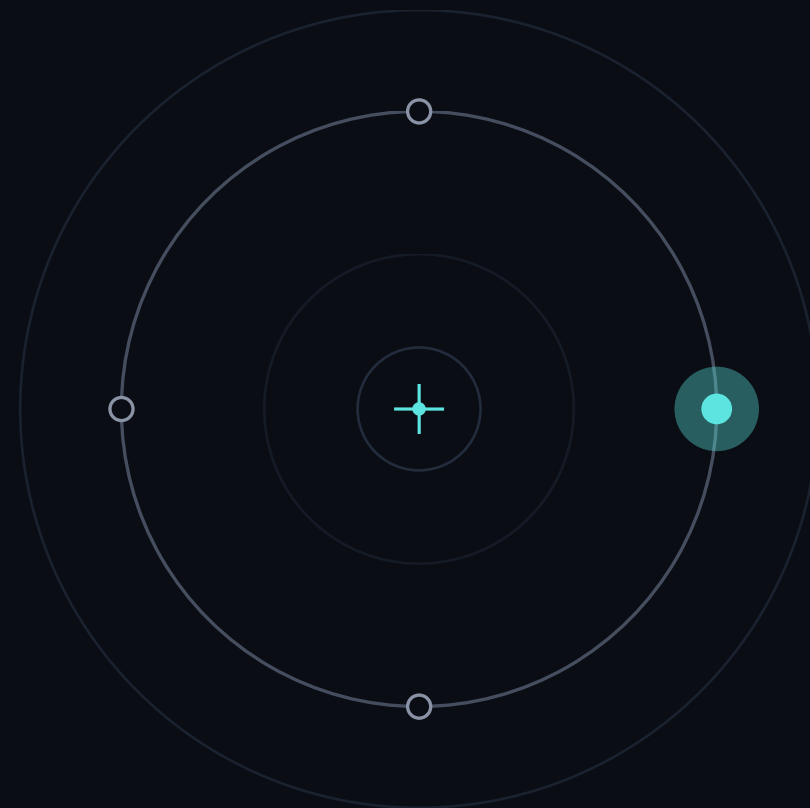
- ✓ 连不上 → 友好提示
- ✓ 超时 → timeout
- ✓ 报错 → 捕获并给备用路径
- ✓ 输出 → 保留 trace

使用 MCP 的成熟度：不是永不失败，而是失败时不崩。

06

Part 6

交付



推荐轨：ResuMatch

- ✓ 接文件 Server 读简历
- ✓ 自建打分 Server
- ✓ 跑出匹配 + 缺口
- ✓ 截图交付

自选轨

- ✓ 课程 RAG：接检索 Server
- ✓ 日程助手：接日历 / 文件 Server
- ✓ 共同点：真数据、真调用
- ✓ 按同一张清单验收

选题可以变，但使用链路不变。

基础档（人人要过）

- ① 接通官方文件 Server
- ② 真读一个文件
- ③ 加只读白名单
- ④ 截图证明 trace

进阶档 ★

- ✓ ① 用 FastMCP 建 Server
- ✓ ② 发布 2 个工具 + 1 个 Resource
- ✓ ③ Inspector 单测通过
- ✓ ④ 接进 Agent 跑通

交付不是 PPT 截图，是能跑的代码 + 真实工具调用截图。

- ✓ ☐ 真的接上 Server：trace 里看得到工具调用。
- ✓ ☐ 真的读了数据：输入来自文件 / 服务，不是手填假数据。
- ✓ ☐ 权限收住了：沙箱 + 白名单。
- ✓ ☐ 失败不崩：连不上、超时、工具报错都有兜底。

验收重点是使用证据，不是概念背诵。

今天能独立完成

- 接一个官方 Server
- 限制权限
- 自建一个小 Server
- 用 trace 验证真实调用

第 6 节 · 多工具协同

- ✓ 接多个 Server
- ✓ 本地 tool + MCP tool 混用
- ✓ 失败时换路径
- ✓ 对应交付：P2 阶段成品

今天先把一条链路跑真；下节再让多条链路协同。